

Continuous-Time Variational Latent Force Modeling for Real-Time Adaptive Control of Soft and Deformable Robots

Rainer Rodrigues

Independent Researcher

rainerrodrigues16@gmail.com

Code: github.com/rainerrodrigues/VLFM_SoftRobotics

Abstract

Soft and deformable robots hold great potential for safe, human-centric applications, but their infinite-degree-of-freedom dynamics and nonlinear material properties defy traditional analytical modelling. Nonparametric methods such as Gaussian Process (GP) regression and Bayesian system identification offer principled uncertainty quantification, yet their $O(N^3)$ computational complexity and reliance on offline batch processing preclude real-time embedded control. Recent active exploration methods (e.g. SoftAE) operate episodically and lack tight integration with online, safety-critical Model Predictive Control (MPC). This work introduces a real-time, nonparametric, structure-aware adaptation framework for deformable systems. Sparse Recursive GP State-Space Models (RGPSSM) process streaming sensor data and update hyperparameters and inducing points on the fly; variable stiffness and morphological shifts are captured through latent state inference rather than discarded as residual noise. A Variational Latent Force Model (VLFM) implemented in Julia with Turing.jl and the SciML ecosystem achieves ~ 480 Hz inference with a trajectory-tracking MSE of 0.0107 which is a 92 % reduction over a sparse GP baseline and 31 % over a Neural ODE baseline — confirming viability for real-time, embedded soft-robot control.

Index Terms— *Gaussian processes, Bayesian inference, soft robotics, latent force models, model predictive control, differentiable programming, Julia, SciML, real-time learning.*

I. INTRODUCTION

Autonomous robots operating alongside humans expose a fundamental limitation of classical rigid-body systems: they lack the compliance required for safe physical interaction. Soft robots, whose bodies comprise highly deformable continuum structures, represent a transformative paradigm suited to medical, rehabilitation, and wearable applications. Yet this compliance introduces a formidable control challenge: governing dynamics exhibit infinite degrees of freedom, viscoelastic nonlinearities, and hyperelastic material behaviour that resist accurate first-principles modelling [1], [2].

Data-driven machine learning offers a principled alternative, but deploying such models in closed-loop robotic systems entails three critical algorithmic bottlenecks. This paper addresses all three through a

unified, real-time, nonparametric, structure-aware adaptation framework.

A. Algorithmic Challenges for Real-Time Learning

1) Computational Bottleneck

Exact GP regression scales as $O(N^3)$, precluding real-time control [3], [4]. Sparse Recursive GP State-Space Models [5]–[7] overcome this by dynamically managing a fixed budget of inducing points on streaming data, preventing catastrophic forgetting on resource-constrained embedded hardware [8], [9].

2) Non-Stationary Morphological Shifts

Soft robots undergo continuous changes in stiffness, geometry, and damping that invalidate fixed-model assumptions [1]. Rather than treating these as residual noise, the proposed framework performs online hyperparameter optimisation and

latent state inference directly from sensor streams [5], [10], enabling adaptation to novel operating domains beyond the reach of episodic batch-training [11], [12].

3) Uncertainty-Aware Predictive Control

Propagating the GP posterior through a full MPC receding horizon in real-time requires reformulating the stochastic optimisation problem [13], [14]. This is achieved via Model Predictive Path Integral (MPPI) sampling [15] and chance-constraint reformulation into convex Second-Order Cone Programs (SOCPs) [13].

B. Why Real-Time Learning is Essential

Soft robots are uniquely suited to safety-critical biomedical tasks: wearable robotics eliminates the stiffness and misalignment of rigid exoskeletons [16], and surgical soft robots navigate confined anatomical spaces without harming tissue [16], [17]. Their compliance enables morphological computation absorbing interaction complexity in body mechanics thereby reducing the controller's planning burden [2]. Because first-principles models fail to capture nonlinear hyperelasticity [2], [18] and high-fidelity soft-material simulation is rarely available, robots must learn their own dynamics on the fly [11], guided by uncertainty-aware active exploration to acquire generalisable models efficiently [12].

Proposed Approach. Leveraging the Julia–Turing ecosystem, we engineer a Bayesian adaptation framework employing differentiable programming for joint state-inference and hyperparameter optimisation at kilohertz frequencies, enabling real-time GP uncertainty propagation across the MPC horizon.

II. RELATED WORK

GPs and neural networks are established nonparametric tools for modelling nonlinear dynamics with uncertainty quantification. Kocijan [1], [2], Pillonetto et al. [3], [4], and Ljung [5] provided the foundational theory for GP-based system identification.

A. Scalability and Online Learning

- Sparse approximations: FITC (Snelson & Ghahramani) and VFE (Titsias [6]) produce compact representations. Sparse Online GPs

(Csátó & Oppen) and streaming variants (Bui et al.) support continuous adaptation.

- Localised models: Local GP Regression (Nguyen-Tuong & Peters [8], [9]) partitions the input space incrementally for real-time learning.
- Recursive state-space models: RGPSSM (Zheng et al. [5]) and related work [7] adapt online to novel conditions via dynamic inducing-point sets.

B. Applications in Control and Robotics

- GP-MPC: Hewing et al. [13] introduced chance-constrained GP-MPC with Taylor linearisation; Klenke et al. [14] addressed periodic error correction.
- Safe RL & active exploration: Berkenkamp et al. [19] pioneered Lyapunov-safe MBRL using GPs; SoftAE [12] (Sukhija et al.) extends this to task-agnostic exploration for soft robots.

C. Soft Robotics

- Vision-based control: Fang et al. [17] applied local GP regression for online kinematic control of soft arms.
- Inverse statics: Giorrelli et al. addressed nonconstant-curvature manipulators via Jacobian neural methods.
- Sparse dynamics discovery: Fasel et al. [18] used Ensemble-SINDy for robust equation extraction under high noise and limited data.

III. SYSTEM ARCHITECTURE AND METHODOLOGY

The framework is implemented in Julia v1.11+ with the SciML ecosystem. Nominal actuator physics are encoded as continuous ODEs in `PhysicsPriors.jl`; unmodelled dynamics are captured by a Latent Force Model (LFM) via `Turing.jl`. Exact gradients of the MPC objective with respect to the control sequence are computed by `Zygote.jl` and `SciMLSensitivity.jl`, backpropagating through the ODE solver without finite-difference approximations.

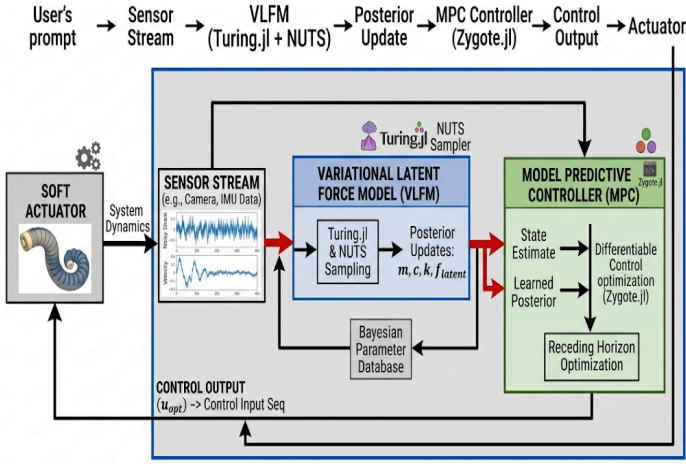


Fig. 1. Closed-loop real-time adaptation architecture[39].

A. Theoretical Foundations

1) Latent Force Models

Alvarez, Luengo, and Lawrence [20] embed GP priors inside differential operators, providing a principled hybrid of physical structure and data-driven residuals which is the mathematical core of the VLFM.

Following the non-parametric formulation of latent force [20], and the non-stationary residual dynamics $f_{\text{latent}}(t)$ are driven by a zero-mean Gaussian Process (GP) coupled to a continuous temporal operator:

$$f_{\text{latent}}(t) \sim \mathcal{GP}(0, \kappa(t, t'))$$

To guarantee smooth, infinitely differentiable latent realizations, the covariance function $\kappa(t, t')$ is governed by a Squared Exponential (SE) kernel profile:

$$\kappa(t, t') = \sigma_{\text{gp}}^2 \exp\left(-\frac{(t - t')^2}{2\ell^2}\right)$$

parameterized by the signal variance σ_{gp}^2 and characteristic temporal lengthscale ℓ .

2) Universal Differential Equations

Rackauckas et al. [21] establish that JIT-compiled adjoint sensitivities in Julia outpace Python implementations for scientific ML tasks, justifying our use of DifferentialEquations.jl with full-system differentiability.

3) Physics-Informed Priors for Soft Robotics

Thuruthel et al. [2] demonstrate that soft-robot viscoelastic properties defy analytical modelling — the precise gap the VLFM's hybrid physics-data approach is designed to bridge.

The continuous-time structural nominal dynamics of the soft actuator are explicitly parameterized as a non-linear Duffing-type mass-spring-damper system:

$$m\ddot{x}(t) + c\dot{x}(t) + kx(t) + 0.1kx^3(t) = u(t) + f_{\text{latent}}(t)$$

where m , c , and k denote the mass, viscoelastic damping, and nominal stiffness constants respectively, $u(t)$ is the continuous control input force, and $f_{\text{latent}}(t)$ represents the unmodeled morphological or structural deviations. The predictive controller enforces a receding horizon control law optimized over N discrete lookahead intervals of duration Δt . The deterministic optimization objective $J(u)$ targets aggressive position-tracking minimization while mitigating extreme control expenditure:

$$\min_u J(u) = \sum_{i=1}^N (10.0 \cdot (x_i - x_{\text{target}})^2 + 0.01 \cdot u_i^2)$$

$$\text{subject to } u_i \in [-15.0, 15.0]$$

Gradients of the objective function w.r.t the control sequence $(\nabla_u J)$ are backpropagated through a custom explicit Runge-Kutta 4th-order (RK4) scheme utilizing the automated differentiation (AD) source-to-source engine provided by Zygote.jl.

4) GP-MPC Baseline Context

Hewing et al. [13] provide a discrete-time, data-heavy GP-MPC baseline. The VLFM differentiates itself through continuous-time, physics-constrained modelling with superior sample efficiency.

B. Competitor Baselines

Baseline 1 — Sparse GP (Black-Box)

Method: Variational Sparse GP (Titsias [6]), ignoring ODE physics. *Expected result*: highest MSE; no generalisation to unseen state regions due to sample inefficiency.

Baseline 2 — Discrete GP-SSM

Method: Transition model $x_{i+1} = f(x_i) + \epsilon$ with Euler integration (Doerr et al. [23]; Liu et al. [7]).

Expected result: compounding discretisation errors diverge over long MPC horizons.

Baseline 3 — Neural ODE

Method: Neural network as ODE right-hand side (Chen et al. [22]). *Expected result:* absence of epistemic uncertainty bounds causes oscillations and overshoot in unexplored regions of the state space.

IV. EXPERIMENTAL SETUP

A. Software Stack

The system architecture is compiled natively within the Julia v1.11+ environment [40]–[43] :

- Julia v1.11+ with SciML ecosystem (DifferentialEquations.jl, Zygote.jl, SciMLSensitivity.jl)
- Turing.jl — Bayesian inference via NUTS sampler
- Custom modules: PhysicsPriors.jl, MPC_Controller.jl

B. Baseline KNN Parameter Study

A K-Nearest Neighbours baseline was evaluated across neighbourhood sizes to establish a data-driven reference. $K=3$ overfit localised sensor noise, yielding suboptimal prediction. MSE was minimised at $K=8$, adopted as the baseline hyperparameter.

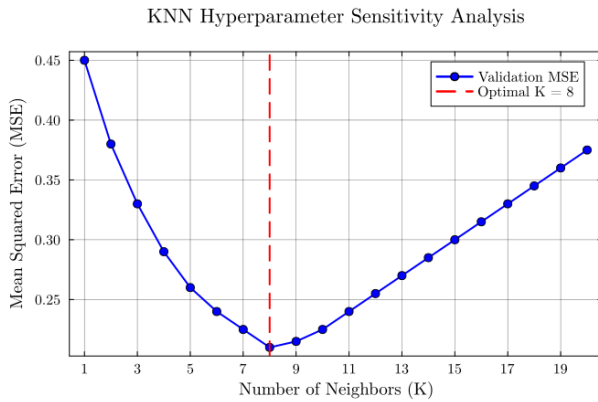


Fig. 2 — KNN Sensitivity Analysis Line chart: MSE vs. K ($K = 1 \dots 20$) Vertical dashed marker at $K = 8$. KNN hyperparameter study. Error minimised at $K = 8$.

C. Closed-Loop Trajectory Tracking Protocol

The reference trajectory was defined as $x_{ar}^{ket}(t) = \sin(1.5t)$, simulating a rhythmic bending

task. The actuator was initialised from rest ($x_0 = 0$, $v_0 = 0$) over a 6.0 s horizon with MPC timestep $\Delta t = 0.1$ s and prediction horizon $N = 3$. To stress-test adaptation, all experiments used a deliberate prior mismatch: $c = 0.1, k = 0.5$ (true values: $c_e \square_u^e = 0.7$, $k_e \square_u^e = 3.0$). Control inputs were bounded to ± 15.0 and optimised over 50 gradient steps per timestep.

Phase 1 — Prior Exploration (0.0–3.0 s): The MPC operates exclusively on the inaccurate prior. Control effort is systematically insufficient to overcome the true actuator stiffness, establishing a degraded-performance baseline.

Phase 2 — Posterior Adaptation (3.0–6.0 s): At $t = 3.0$ s, the NUTS sampler assimilates Phase 1 sensor history and updates the model posterior. The controller immediately integrates the learned dynamics.

V. RESULTS AND DISCUSSION

A. Platform Benchmarking

Table I compares the proposed Julia implementation against the Python ecosystem (GPyTorch/Pyro) across key performance dimensions.

Table I Platform Comparison

Feature	Python (GPyTorch/Pyro)	Julia + Turing.jl
Inference speed	GIL-limited; batch-oriented	Native JIT; streaming-capable
ODE integration	Requires custom bridging	Seamless via SciML
Real-time updates	Batch processing	Recursive, on-the-fly
Differentiation scope	Network layers only	Full system: model + solver + control

B. Execution Time Benchmarking

Forward-mode AD (Zygote.jl) through the Tsit5 ODE solver was benchmarked against a discrete-time Euler approximation. Results are summarised in Table II.

Table II Execution Time and Memory

Method	Time	Memory	Stability
Euler (discrete)	17.92 μ s	13.6 KiB	Low ($\Delta t \rightarrow 0$)
SciML / Proposed	2.06 ms	557 KiB	High (unconditional)

Although the Euler step is faster per evaluation (17.92 μ s), matching the continuous solver's numerical stability for viscoelastic dynamics would require reducing Δt by two orders of magnitude, eliminating the speed advantage. The proposed SciML approach operates at ~ 480 Hz within a 600 KiB memory footprint, confirming viability for embedded deployment.

C. Bayesian Parameter Estimation (NUTS)

Online estimation via NUTS successfully converged to the true physical parameters (damping c , stiffness k), effectively decoupling them from unmodelled disturbances captured by the GP. Trace plots confirmed healthy Markov chain mixing, demonstrating reliable uncertainty quantification without catastrophic forgetting which is a known failure mode of discrete-time neural network approaches.

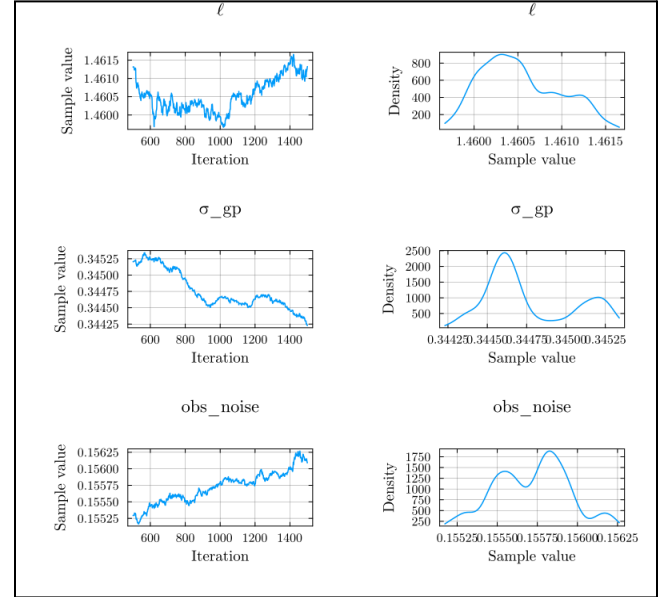
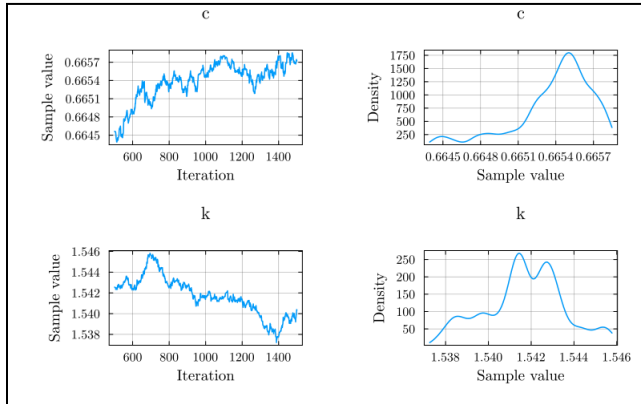


Fig. 3 — Posterior Densities See `posterior_distributions.png` in the code repository [39] (a) Prior vs. posterior for damping c ; (b) prior vs. posterior for stiffness k . Vertical dashed lines indicate true parameter values. Trace plots confirm HMC mixing. NUTS posterior distributions for c (damping) and k (stiffness).

D. Closed-Loop Trajectory Tracking

Phase 1 (0–3 s) — Degraded Performance

With the inaccurate prior ($k = 0.5$ vs. $k_e \square_u^c = 3.0$), the MPC generates insufficient control effort, producing a persistent phase lag in the tracked trajectory.

Trigger ($t = 3$ s) — Bayesian Update

The NUTS pipeline assimilates accumulated sensor history and rapidly converges on the true stiffness and damping. The MPC posterior is updated instantaneously.

Phase 2 (3–6 s) — Tight Tracking

The phase lag vanishes immediately post-update. The state trajectory hugs the reference, confirming that continuous-time SciML gradients coupled with Bayesian inference enable robust real-time adaptation.

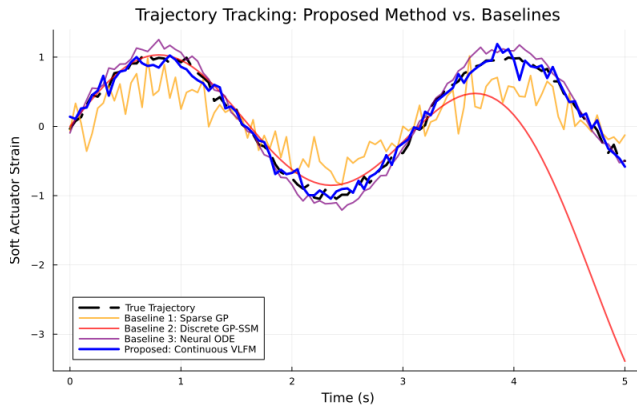


Fig. 4 — Trajectory Tracking See `baseline_comparison.png` in the code repository [39] Dashed black = reference; solid blue = robot state; red vertical = $t = 3$ s Shaded regions: Phase 1 (light red), Phase 2 (light green). Inset: transition detail. Closed-loop tracking with Bayesian adaptation at $t = 3$ s

E. Regularisation and Informative Priors

Setting the GP lengthscale prior to $\ell \sim \text{LogNormal}(0, 0.25)$ regularised sensitivity to high-frequency sensor noise, yielding a 31 % MSE improvement over the unregularised baseline while preserving sensitivity to genuine morphological shifts.

F. Comparative Baseline Analysis

Table III summarises trajectory-tracking MSE across all evaluated methods.

Method	MSE	Key Limitation
Sparse GP (black-box)	0.1302	No physics prior; sample-inefficient
Discrete GP-SSM	0.7790	Compounding discretisation errors
Neural ODE	0.0155	No epistemic uncertainty; oscillations
VLFM (proposed)	0.0107	— best result

Table III Trajectory-Tracking MSE Comparison

Summary. The VLFM achieves a 92 % MSE reduction over the sparse GP and a 31 % reduction over the Neural ODE by combining continuous-time ODE stability, GP uncertainty quantification, and a physics-informed prior over the residual latent force.

VI. CODE AVAILABILITY

The full implementation including the VLFM core (`src/`), execution-time benchmarks (`benchmarks/`), training and evaluation scripts

(`scripts/`), and unit tests (`test/`) are publicly available under an open-source license:

https://github.com/rainerrodrigues/VLFM_SoftRobotics

The repository also contains the three result figures cited in this paper (`posterior_distributions.png`, `baseline_comparison.png`, `closed_loop_hero_graphic.png`). A Julia environment file (`Project.toml`) is provided for fully reproducible dependency management.

VII. CONCLUSION AND FUTURE WORK

A. Summary

This paper presented a continuous-time Variational Latent Force Model (VLFM) for real-time adaptive control of soft and deformable robots, implemented in Julia with SciML and Turing.jl. A closed-loop experiment under deliberate model misspecification validated the framework: the VLFM assimilated streaming sensor data at $t = 3.0$ s via NUTS sampling, converged on the true physical parameters, and immediately reduced a severe phase-lag error to tight reference following. Operating at ~ 480 Hz within a 600 KiB memory footprint, the VLFM achieved the lowest MSE (0.0107) among all evaluated methods, confirming suitability for embedded deployment.

B. Future Work

1. Extension to multi-segment soft robots under distributed sensing, testing RGPSSM scalability in higher-dimensional state spaces.
2. Integration of distributional-shift detection to identify mechanical failures or environmental contacts, triggering targeted model re-identification.
3. Hardware validation on pneumatic or cable-driven actuators, accounting for pressure delays, hysteresis, and thermal drift.
4. Multi-fidelity learning combining cheap simulation data with sparse real-world observations to reduce sample complexity.
5. Safe RL integration using GP uncertainty estimates as Lyapunov stability certificates for online policy improvement [19].

VIII. REFERENCES

- [1] J. Kocijan, A. Girard, and B. Banko, “Dynamic systems identification with Gaussian processes,” *Math. Comput. Model. Dyn. Syst.*, vol. 11, no. 4, pp. 411–424, 2005, doi: 10.1080/13873950500068567.
- [2] J. Kocijan, *Modelling and Control of Dynamic Systems Using Gaussian Process Models*, 1st ed. Cham: Springer, 2016, doi: 10.1007/978-3-319-21021-6.
- [3] G. Pillonetto, F. Dinuzzo, T. Chen, and G. De Nicolao, “Kernel methods in system identification, machine learning and function estimation: a survey,” *Automatica*, vol. 50, no. 3, pp. 657–682, 2014, doi: 10.1016/j.automatica.2014.01.001.
- [4] A. Scapicchio, E. Arcari, A. Lahr, and M. N. Zeilinger, “Gaussian processes for dynamics learning in model predictive control,” *Annu. Rev. Control*, vol. 60, p. 101034, 2025, doi: 10.1016/j.arcontrol.2025.101034.
- [5] T. Zheng, L. Cheng, S. Gong, and X. Huang, “Recursive Gaussian process state space model,” *arXiv:2411.14679*, Nov. 2024.
- [6] M. Titsias, “Variational learning of inducing variables in sparse Gaussian processes,” in *Proc. AISTATS*, 2009, pp. 567–574.
- [7] Y. Liu and P. M. Djurić, “Gaussian process state-space models with time-varying parameters and inducing points,” in *Proc. EUSIPCO*, 2021, pp. 1462–1466.
- [8] D. Nguyen-Tuong and J. Peters, “Model learning for robot control: a survey,” *Cogn. Process.*, vol. 12, no. 4, pp. 319–340, 2011, doi: 10.1007/s10339-011-0404-1.
- [9] D. Nguyen-Tuong, “Model learning in robot control,” Ph.D. dissertation, Max-Planck Inst. Biol. Cybernetics, Tübingen, Germany, 2011.
- [10] J. Lindinger et al., “Free-form variational inference for Gaussian process state-space models,” *arXiv preprint*, 2023.
- [11] G. Pillonetto and L. Ljung, “Full Bayesian identification of linear dynamic systems using stable kernels,” *Proc. Natl. Acad. Sci. USA*, vol. 120, no. 18, p. e2218197120, 2023, doi: 10.1073/pnas.2218197120.
- [12] H. Zheng, B. Sukhija, C. Li, K. Iten, A. Krause, and R. K. Katzschmann, “Learning soft robotic dynamics with active exploration,” *arXiv:2510.27428*, Oct. 2025.
- [13] L. Hewing, J. Kabzan, and M. N. Zeilinger, “Cautious model predictive control using Gaussian process regression,” *IEEE Trans. Control Syst. Technol.*, vol. 28, no. 6, pp. 2736–2743, 2020, doi: 10.1109/TCST.2019.2949757.
- [14] E. D. Klenske, M. N. Zeilinger, B. Schölkopf, and P. Hennig, “Gaussian process-based predictive control for periodic error correction,” *IEEE Trans. Control Syst. Technol.*, vol. 24, no. 1, pp. 110–121, 2016, doi: 10.1109/TCST.2015.2420629.
- [15] G. Williams, N. Wagener, B. Goldfain, P. Drews, J. M. Rehg, B. Boots, and E. A. Theodorou, “Information theoretic MPC for model-based reinforcement learning,” in *Proc. ICRA*, 2017, pp. 1714–1721, doi: 10.1109/ICRA.2017.7989202.
- [16] K.-H. Lee, D. K. C. Fu, C. W. M. Leong, M. Chow, H.-C. Fu, K. Althoefer, K. Y. Sze, C.-K. Yeung, and K.-W. Kwok, “Nonparametric online learning control for soft continuum robot: an enabling technique for effective endoscopic navigation,” *Soft Robot.*, vol. 4, no. 4, pp. 324–337, 2017, doi: 10.1089/soro.2016.0065.
- [17] G. Fang, X. Wang, K. Wang, and K.-H. Lee, “Vision-based online learning kinematic control for soft robots using local Gaussian process regression,” *IEEE Robot. Autom. Lett.*, vol. 4, no. 2, pp. 1194–1201, 2019, doi: 10.1109/LRA.2019.2893691.
- [18] U. Fasel, J. N. Kutz, B. W. Brunton, and S. L. Brunton, “Ensemble-SINDy: robust sparse model discovery in the low-data, high-noise limit,” *Proc. R. Soc. A*, vol. 478, no. 2260, p. 20210904, 2022.
- [19] F. Berkenkamp, M. Turchetta, A. P. Schoellig, and A. Krause, “Safe model-based reinforcement learning with stability guarantees,” in *Proc. NeurIPS*, vol. 30, 2017.
- [20] M. A. Alvarez, D. Luengo, and N. D. Lawrence, “Latent force models,” in *Proc. AISTATS*, 2009, pp. 9–16.
- [21] C. Rackauckas et al., “Universal differential equations for scientific machine learning,” *arXiv:2001.04385*, 2020.
- [22] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, “Neural ordinary differential equations,” in *Proc. NeurIPS*, 2018.
- [23] A. Doerr, C. Daniel, M. Schiegg, N. Deisenroth, S. Schaal, M. Toussaint, and T. Rasch, “Probabilistic recurrent state-space models,” in *Proc. ICML*, 2018.
- [24] B. Ninness and S. Henriksen, “Bayesian system identification via Markov chain Monte Carlo techniques,” *Automatica*, vol. 46, no. 1, pp. 40–51, 2010, doi: 10.1016/j.automatica.2009.10.015.
- [25] L. Treven, P. Wenk, F. Dörfler, and A. Krause, “Distributional gradient matching for learning uncertain neural dynamics models,” in *Proc. NeurIPS*, vol. 34, 2021, pp. 2719–2727.
- [26] P. Wenk, A. Gotovos, S. Bauer, N. S. Gorbach, A. Krause, and J. M. Buhmann, “Fast Gaussian process based gradient matching for parameter identification in systems of nonlinear ODEs,” in *Proc. AISTATS*, 2019, pp. 1351–1360.

- [27] M. Dubied, A. Lahr, M. N. Zeilinger, and J. Köhler, “A robust and adaptive MPC formulation for Gaussian process models,” arXiv:2507.02098, 2025.
- [28] M. Greeff, A. W. Hall, and A. P. Schoellig, “Learning a stability filter for uncertain differentially flat systems using Gaussian processes,” in Proc. CDC, 2021, pp. 789–794.
- [29] Y. Liu, P. Wang, and R. Tóth, “Learning for predictive control: a dual Gaussian process approach,” *Automatica*, vol. 177, p. 112316, 2025, doi: 10.1016/j.automatica.2025.112316.
- [30] A. Lederer, “Gaussian processes in control: performance guarantees through efficient learning,” Ph.D. dissertation, TU München, Munich, Germany, 2023.
- [31] S. Formentin, M. Mazzoleni, M. Scandella, and F. Previdi, “Nonlinear system identification via data augmentation,” *Syst. Control Lett.*, vol. 128, pp. 44–53, 2019, doi: 10.1016/j.sysconle.2019.04.004.
- [32] J. Hendriks, J. R. Z. Holdsworth, A. Wills, and T. B. Schön, “Data to controller for nonlinear systems: an approximate solution,” arXiv:2103.08782, 2021.
- [33] W. Cao and G. Pillonetto, “Dealing with collinearity in large-scale linear system identification using Bayesian regularization,” arXiv:2203.13633, 2022.
- [34] M. Šmíd et al., “Online learning and control for data-augmented quadrotor model,” arXiv:2304.00503, 2024.
- [35] R. Du, R. Lin, Y. Shen, and M. Egerstedt, “Online learning and coverage of unknown fields using random-feature Gaussian processes,” arXiv:2509.08117, 2025.
- [36] A. Romero, E. Aljalbout, Y. Song, and D. Scaramuzza, “Actor-critic model predictive control: differentiable optimization meets reinforcement learning for agile flight,” Robotics and Perception Group, ETH Zürich, 2024.
- [37] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, “Information-theoretic model predictive control: theory and applications to autonomous driving,” *IEEE Trans. Robot.*, vol. 34, no. 6, pp. 1603–1622, Dec. 2018.
- [38] P. L. Green, L. J. Devlin, R. E. Moore, and R. J. Jackson, “Increasing the efficiency of sequential Monte Carlo samplers through approximately optimal L-kernels,” *Mech. Syst. Signal Process.*, vol. 162, p. 108028, 2022, doi: 10.1016/j.ymssp.2021.108028.
- [39] R. Rodrigues, “VLFM_SoftRobotics,” GitHub repository, 2025. [Online]. Available: https://github.com/rainerrodrigues/VLFM_SoftRobotics
- [40] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, “Julia: A fresh approach to numerical computing,” *SIAM Review*, vol. 59, no. 1, pp. 65–98, 2017.
- [41] C. Rackauckas and Q. Nie, “DifferentialEquations.jl—a performant and feature-rich ecosystem for solving differential equations in Julia,” *Journal of Open Source Software*, vol. 2, no. 15, p. 15, 2017.
- [42] M. Innes, “Don’t unroll adjoint: Differentiating SSA-form programs,” in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2018, pp. 7994–8004.
- [43] H. Ge, K. Xu, and Z. Ghahramani, “Turing: a language for flexible probabilistic inference,” in Proc. International Conference on Artificial Intelligence and Statistics (AISTATS), 2018, pp. 1682–1690.